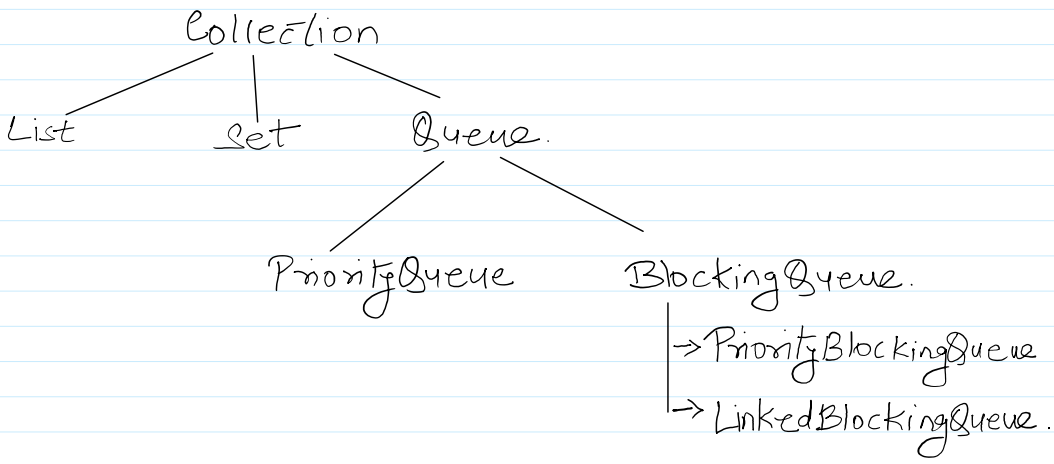
Enhancement in Collection

01 June 2025 10:34



Queue(I) Java 1.5 Version
It is a child interface of Collection interface. If we want to represent a group of individual objects prior to processing, then we should go for Queue. For example, before sending SMS messages to all mobile numbers that we have store in some data structure. in which order we add mobile number in the same order messages should be delivered for this (FIFO) requirement Queue is the best choice.

Usually Queue follows (FIFO) but based on our requirement we can implement our own Priority Queue.
From Java1.5 version onwards LinkedList class also implements Queue interface. LinkedList based implementation of queue always follow FIFO.

Queue interface Specific methods:
boolean offer(Object o) ==> to add object into the queue

Object poll() ==> to remove and return head element of queue.

Object peak() ==> it returns the head element of queue. If queue is empty then this method returns null.

Object element() ==> it returns head element of the queue. If queue is empty then this method raises runtime exception i.e., "NoSuchElementException"

Object remove() ==> It returns the head element of the queue. If queue is empty, then this method raises runtime exception i.e., "NoSuchElementException"

Official Documentation:
public interface Queue<E> extends Collection<E>
A collection designed for holding elements prior to processing. Besides basic Collection operations, queues provide additional insertion, extraction, and inspection operations. Each of these methods exists in two forms: one throws an exception if the operation fails, the other returns a special value (either null or false, depending on the operation). The latter form of the insert operation is designed specifically for use with capacity-restricted Queue implementations; in most implementations, insert operations cannot fail.

	Throws exception	*Returns special value*
Insert	add(e)	offer(e)
Remove	remove()	poll()
Examine	element()	peek()

Summary of Queue methods
Queues typically, but do not necessarily, order elements in a FIFO (first-in-first-out) manner. Among the exceptions are priority queues, which order elements according to a supplied comparator, or the elements' natural ordering, and LIFO

queues (or stacks) which order the elements LIFO (last-in-first-out). Whatever the ordering used, the *head* of the queue is that element which would be removed by a call to remove() or poll(). In a FIFO queue, all new elements are inserted at the *tail* of the queue. Other kinds of queues may use different placement rules. Every Queue implementation must specify its ordering properties.
The offer method inserts an element if possible, otherwise returning false. This differs from the Collection.add method, which can fail to add an element only by throwing an unchecked exception. The offer method is designed for use when failure is a normal, rather than exceptional occurrence, for example, in fixed-capacity (or "bounded") queues.
The remove() and poll() methods remove and return the head of the queue. Exactly which element is removed from the queue is a function of the queue's ordering policy, which differs from implementation to implementation. The remove() and poll() methods differ only in their behavior when the queue is empty: the remove() method throws an exception, while the poll() method returns null.
The element() and peek() methods return, but do not remove, the head of the queue.
The Queue interface does not define the *blocking queue methods*, which are common in concurrent programming. These methods, which wait for elements to appear or for space to become available, are defined in the BlockingQueue interface, which extends this interface.
Queue implementations generally do not allow insertion of null elements, although some implementations, such as LinkedList, do not prohibit insertion of null. Even in the implementations that permit it, null should not be inserted into a Queue, as null is also used as a special return value by the poll method to indicate that the queue contains no elements.
Queue implementations generally do not define element-based versions of methods equals and hashCode but instead inherit the identity based versions from class Object, because element-based equality is not always well-defined for queues with the same elements but different ordering properties.
This interface is a member of the Java Collections Framework.

Since:

1.5

From <https://docs.oracle.com/en/java/javase/24/docs/api//java.base/java/util/Queue.html>

All Methods | Instance Methods | Abstract Methods

Modifier and Type	Method	Description
boolean	add(E e)	Inserts the specified element into this queue if it is possible to do so immediately without violating capacity restrictions, returning true upon success and throwing an IllegalStateException if no space is currently available.
E	element()	Retrieves, but does not remove, the head of this queue.
boolean	offer(E e)	Inserts the specified element into this queue if it is possible to do so immediately without violating capacity restrictions.
E	peek()	Retrieves, but does not remove, the head of this queue, or returns null if this queue is empty.
E	poll()	Retrieves and removes the head of this queue, or returns null if this queue is empty.
E	remove()	Retrieves and removes the head of this queue.

PriorityQueue

- If we want to represent a group of individual objects prior to processing, according to some priority, then we should go for priority queue.
- The priority can be either default natural sorting order or customized sorting order defined by Comparator.
- Insertion order is not preserved and it is based on some priority.
- Duplicate objects are not allowed.
- If we are depending on default natural sorting order, it is compulsory objects should be homogeneous and comparable. Otherwise we will get runtime exception, saying **ClassCastException**. If we are defining our own sorting by comparator, then objects need not be homogeneous and Comparable.
- "null" is not allowed at all.

Constructors

PriorityQueue pq = new PriorityQueue(); ==> It creates an empty priority queue with default initial capacity 11 and all objects is inserted according to default natural sorting order.

PriorityQueue pq = new PriorityQueue(int initialCapacity); ==> It creates an empty priority queue with initial capacity passed as parameter and all objects is inserted according to default natural sorting order.

PriorityQueue pq = new PriorityQueue(int initialCapacity, Comparator c); ==> It creates an empty priority queue with initial capacity passes as parameter and all objects is inserted according to customized sorting order by the Comparator c.

PriorityQueue pq = new PriorityQueue(SortedSet s);

PriorityQueue pq = new PriorityQueue(Collection c);

→ interconversion constructor.

Official Documentation:

public class PriorityQueue<E> extends AbstractQueue<E> implements Serializable

An unbounded priority queue based on a priority heap. The elements of the priority queue are ordered according to their natural ordering, or by a Comparator provided at queue construction time, depending on which constructor is used. A priority queue does not permit null elements. A priority queue relying on natural ordering also does not permit insertion of non-comparable objects (doing so may result in ClassCastException).

The *head* of this queue is the *least* element with respect to the specified ordering. If multiple elements are tied for least value, the head is one of those elements -- ties are broken arbitrarily. The queue retrieval operations poll, remove, peek, and element access the element at the head of the queue.

A priority queue is unbounded, but has an internal *capacity* governing the size of an array used to store the elements on the queue. It is always at least as large as the queue size. As elements are added to a priority queue, its capacity grows automatically. The details of the growth policy are not specified.

This class and its iterator implement all of the *optional* methods of the Collection and Iterator interfaces. The Iterator provided in method iterator() and the Spliterator provided in method spliterator() are *not* guaranteed to traverse the elements of the priority queue in any particular order. If you need ordered traversal, consider using Arrays.sort(pq.toArray()).

Note that this implementation is not synchronized. Multiple threads should not access a PriorityQueue instance concurrently if any of the threads modifies the queue. Instead, use the thread-safe PriorityBlockingQueue class.

Implementation note: this implementation provides O(log(n)) time for the enqueuing and dequeuing methods (offer, poll, remove() and add); linear time for the remove(Object) and contains(Object) methods; and constant time for the retrieval methods (peek, element, and size).

This class is a member of the Java Collections Framework.

Since:

1.5

From <https://docs.oracle.com/en/java/javase/24/docs/api//java.base/java/util/PriorityQueue.html>

Constructor Summary

Constructors

Constructor	Description
`PriorityQueue()`	Creates a `PriorityQueue` with the default initial capacity (11) that orders its elements according to their natural ordering.
`PriorityQueue(int initialCapacity)`	Creates a `PriorityQueue` with the specified initial capacity that orders its elements according to their natural ordering.
`PriorityQueue(int initialCapacity, Comparator<? super E> comparator)`	Creates a `PriorityQueue` with the specified initial capacity that orders its elements according to the specified comparator.
`PriorityQueue(Collection<? extends E> c)`	Creates a `PriorityQueue` containing the elements in the specified collection.
`PriorityQueue(Comparator<? super E> comparator)`	Creates a `PriorityQueue` with the default initial capacity and whose elements are ordered according to the specified comparator.
`PriorityQueue(PriorityQueue<? extends E> c)`	Creates a `PriorityQueue` containing the elements in the specified priority queue.
`PriorityQueue(SortedSet<? extends E> c)`	Creates a `PriorityQueue` containing the elements in the specified sorted set.

Prof Partha Pratim Chakraborty

Method Summary

All Methods | Instance Methods | Concrete Methods

Modifier and Type	Method	Description
boolean	add(E e)	Inserts the specified element into this priority queue.
void	clear()	Removes all of the elements from this priority queue.
Comparator<? super E>	comparator()	Returns the comparator used to order the elements in this queue, or null if this queue is sorted according to the natural ordering of its elements.
boolean	contains(Object o)	Returns true if this queue contains the specified element.
void	forEach(Consumer<? super E> action)	Performs the given action for each element of the Iterable until all elements have been processed or the action throws an exception.
Iterator<E>	iterator()	Returns an iterator over the elements in this queue.
boolean	offer(E e)	Inserts the specified element into this priority queue.
E	peek()	Retrieves, but does not remove, the head of this queue, or returns null if this queue is empty.
E	poll()	Retrieves and removes the head of this queue, or returns null if this queue is empty.
boolean	remove(Object o)	Removes a single instance of the specified element from this queue, if it is present.
boolean	removeAll(Collection<?> c)	Removes all of this collection's elements that are also contained in the specified collection (optional operation).
boolean	removeIf(Predicate<? super E> filter)	Removes all of the elements of this collection that satisfy the given predicate (optional operation).
boolean	retainAll(Collection<?> c)	Retains only the elements in this collection that are contained in the specified collection (optional operation).
int	size()	Returns the number of elements in this collection.
final Spliterator<E>	spliterator()	Creates a *late-binding* and *fail-fast* Spliterator over the elements in this queue.
Object[]	toArray()	Returns an array containing all of the elements in this queue.
<T> T[]	toArray(T[] a)	Returns an array containing all of the elements in this queue; the runtime type of the returned array is that of the specified array.

```
package Chapter6;

import java.util.*;

public class PriorityQueueDemo {
   public static void main(String[] args) {
      PriorityQueue q = new PriorityQueue();
      System.out.println(q.peek());
      System.out.println(q.element());
   }
}
```

because queue is empty.

Output:

peek() method.

```
null
Exception in thread "main" java.util.NoSuchElementException
        at java.base/java.util.AbstractQueue.element(AbstractQueue.java:136)
        at Chapter6.PriorityQueueDemo.main(PriorityQueueDemo.java:9)
PS D:\Students\Students\SubhalakshmiUEM\Coding\AdvancedJava>
```

Code:

```
package Chapter6;
```

```java
import java.util.*;
public class PriorityQueueDemo {
    public static void main(String[] args) {
        PriorityQueue q = new PriorityQueue();
        // System.out.println(q.peek());
        // System.out.println(q.element());
        for (int i = 0; i < 10; i++) {
            q.offer(i);
        }
        System.out.println(q);
        System.out.println(q.poll());
        System.out.println(q);
        System.out.println(q.poll());
        System.out.println(q);
    }
}
```

Output:

```
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
0
[1, 3, 2, 7, 4, 5, 6, 9, 8]
1
[2, 3, 5, 7, 4, 8, 6, 9]
```

```java
package Chapter6;
import java.util.*;
public class PriorityQueueDemo {
    public static void main(String[] args) {
        PriorityQueue<Integer> q = new PriorityQueue<Integer>();
        Random r = new Random();
        // System.out.println(q.peek());
        // System.out.println(q.element());
        for (int i = 0; i < 10; i++) {
            q.offer(r.nextInt(10, 100));
        }
        System.out.println(q);
        System.out.println(q.poll());
        System.out.println(q);
        System.out.println(q.poll());
        System.out.println(q);
        for(; q.size() > 0; ){
            System.out.println(q.poll());
        }
    }
}
```

maintains the order.

OUTPUT:

```
[12, 31, 22, 31, 61, 99, 49, 82, 39, 82]
12
[22, 31, 49, 31, 61, 99, 82, 82, 39]
22
[31, 31, 49, 39, 61, 99, 82, 82]
31
31
39
49
61
82
82
99
```

Code by using Comparator:

```java
package Chapter6;
import java.util.PriorityQueue;
public class PriorityQueueComparator {
    public static void main(String[] args) {
        PriorityQueue q = new PriorityQueue<>(new MyComparator());
```

```java
        q.add("Aditya");
        q.add("Anchit");
        q.add("Aadya");
        q.add("Aadit");
        q.add("Anant");
        System.out.println(q);
        System.out.println(q.poll());
        System.out.println(q);
        while(q.size() > 0){
            System.out.println(q.poll());
        }
    }
}
```

while polling ordering is maintained.

```java
package Chapter6;
import java.util.Comparator;
public class MyComparator implements Comparator {
    @Override
    public int compare(Object o1, Object o2) {
        String st1 = (String)o1;
        String st2 = (String)o2;
        return st2.compareTo(st1);
    }
}
```

Output:

while storing insertion order is not preserved.

```
[Anchit, Anant, Aadya, Aadit, Aditya]
Anchit
[Anant, Aditya, Aadya, Aadit]
Anant
Aditya
Aadya
Aadit
```

← ordering is customized order.